Лабораторная работа №10
Работа с бинарными файлами

1 Цель работы

1.1 Изучить процесс обработки двоичных файлов средствами языка C# с использованием потоков.

2 Литература

2.1 Ашарина, И. В. Объектно-ориентированное программирование в C++: лекции и упражнения. Учебное пособие для вузов. – 2-е изд., перераб. и доп. Москва : Горячая Линия–Телеком, 2017. – URL: https://ibooks.ru/bookshelf/359752/reading. – Режим доступа: для зарегистрир. пользователей. – Текст : электронный. – гл.24

2.2 Немцова, Т. И. Программирование на языке высокого уровня. Программирование на языке C++ / Т. И. Немцова, С. Ю. Голова, А. И. Терентьев. – Москва : Форум, 2019. – URL: https://ibooks.ru/reading.php?productid=361544. – Режим доступа: для зарегистрир. пользователей. – Текст : электронный. – п.19.8-19.11.

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Написать программу, запрашивающую у пользователя имя файла и записывающую в него n случайных целых чисел от 1 до 100 (n вводится пользователем) в **бинарном** виде.

5.2 Написать программу, запрашивающую у пользователя имя **бинарного** файла из задания п.5.1, считывающую из него целые числа до конца файла и выводящую эти числа и их сумму на экран.

5.3 Написать программу, запрашивающую у пользователя имя **бинарного** файла из задания п.5.1 и номер элемента. Вычислить смещение, перейти к нужному числу и вывести его, не считывая предшествующие числа и не загружая файл целиком.

5.4 Написать программу, запрашивающую имя **бинарного** файла из задания п.5.1 и номера первого и последнего чисел диапазона. Перейти непосредственно к началу диапазона и вычислить сумму его элементов, не считывая предшествующие числа.

5.5 Написать программу для замены одного числа в двоичном файле задания 5.1. Пользователь вводит имя **бинарного** файла, порядковый номер k и новое значение числа. После проверки формата и номера прочитать и вывести старое значение, затем заменить его по вычисленному смещению.

Использовать произвольный доступ и режим открытия существующего файла с чтением и записью; не пересоздавать файл и не переписывать его целиком. Для проверки замены прочитать числа из файла и вывести их на экран

6 Порядок выполнения работы

6.1 Используя Microsoft Visual Studio, создать проект C# и выполнить задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 В чем преимущества использования двоичных файлов?

8.2 С помощью каких функций можно записывать информацию в двоичные файлы?

8.3 С помощью каких функций можно считывать информацию из двоичных файлов?

8.4 Как считать переменные стандартных типов данных из двоичного файла?

8.5 Как считать переменные структурированных типов данных из двоичного файла?

9 Приложение

9.1 Общие сведения о бинарных файлах

Бинарный (двоичный) файл хранит информацию в виде последовательности байтов. В отличие от текстовых файлов, значения в нём записываются во внутреннем двоичном представлении, а не в виде символов.

Для работы с бинарными файлами в C# используются классы из пространства имён System.IO:

FileStream – открывает файл и предоставляет доступ к его содержимому.

BinaryWriter – записывает данные различных типов в бинарный файл.

BinaryReader – считывает данные из бинарного файла.

Для подключения необходимых классов используется директива:

```
using System.IO;
```

9.2 Запись данных с помощью BinaryWriter

Для записи информации необходимо создать файловый поток FileStream, а затем передать его объекту BinaryWriter.

```
using FileStream fs = new FileStream(
    "data.bin",
    FileMode.Create,
    FileAccess.Write
```

```
);

using BinaryWriter writer = new BinaryWriter(fs);

writer.Write(25);
writer.Write(3.14);
writer.Write(true);
```

Метод Write() записывает значение в файл в соответствии с его типом.

9.3 Чтение данных с помощью BinaryReader

Для чтения информации используется класс BinaryReader.

```
using FileStream fs = new FileStream(
    "data.bin",
    FileMode.Open,
    FileAccess.Read
);

using BinaryReader reader = new BinaryReader(fs);

int a = reader.ReadInt32();
double b = reader.ReadDouble();
bool c = reader.ReadBoolean();
```

9.4 Определение размера файла и проверка конца файла

Свойство Length класса FileStream возвращает размер файла в байтах, а свойство Position – текущую позицию файлового указателя.

```
long size = fs.Length;
long position = fs.Position;
```

Для проверки наличия непрочитанных данных можно использовать условие:

```
fs.Position < fs.Length
```

При достижении конца файла попытка прочитать очередное значение методом BinaryReader может вызвать исключение EndOfStreamException.

9.5 Произвольный доступ к файлу

Произвольный доступ позволяет переместить файловый указатель непосредственно к нужному участку файла, не считывая предыдущие данные.

Для перемещения используется метод Seek():

```
fs.Seek(offset, SeekOrigin.Begin);
```

Здесь offset – смещение в байтах, а SeekOrigin.Begin означает отсчёт от начала файла.

Возможные значения SeekOrigin:

Значение	**Начало отсчёта**
Begin	Начало файла
Current	Текущая позиция
End	Конец файла

Если файл состоит из элементов одинакового размера, смещение элемента

можно вычислить по формуле:

Для определения размера типа int можно использовать:

```
int size = sizeof(int);
```

9.6 Изменение данных в существующем файле

Для изменения содержимого файла без его пересоздания необходимо открыть его с правами чтения и записи:

```
using FileStream fs = new FileStream(
    "data.bin",
    FileMode.Open,
    FileAccess.ReadWrite
);
```

Для такого потока допускается использование BinaryReader и BinaryWriter. Перед чтением или записью можно переместить позицию файлового указателя с помощью Seek().

При замене данных следует учитывать, что записываемое значение должно занимать столько же байтов, сколько исходное, если необходимо сохранить структуру файла.

Важно: не используйте FileMode.Create для изменения существующего файла, поскольку этот режим удаляет его прежнее содержимое.